# Sliding Window & Two Pointers: The LeetCode Mastery Plan

## The Golden Rule: Avoiding O(N^2)

- **Use Two Pointers** when you need to search pairs in a sorted array, or when you are modifying an array in-place (comparing a 'reader' pointer and a 'writer' pointer).
- **Use a Sliding Window** when you are asked to find a **contiguous subarray or substring** based on certain conditions (e.g., "maximum sum of size K", "longest string with no repeats").

## Level 1: The Warm-Up (Two Pointers)

**LeetCode 283: Move Zeroes**

- **The Goal:** Given an array, move all `0`s to the end while keeping the non-zero numbers in their original order. You must do this in-place!
- **Why do it:** This teaches you the "Reader/Writer" two-pointer technique. One pointer scans the array, the other points to where the next valid item should be written. It avoids the buggy nested `while` loops!

### Python Solution

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        # 'left' is our writer pointer. It tracks where the next non-zero shoul
        left = 0

        # 'right' is our reader pointer. It scans every item.
        for right in range(len(nums)):
            if nums[right] != 0:
                # Swap the non-zero number to the 'left' position
                nums[left], nums[right] = nums[right], nums[left]
                # Move the writer pointer forward
                left += 1
```

## Level 2: Fixed Sliding Window

**LeetCode 643: Maximum Average Subarray I**

- **The Goal:** Find a contiguous subarray of exactly length `k` that has the maximum average value.
- **Why do it:** The brute force way is to recalculate the sum of every block of `k` numbers. The "Sliding Window" way is to calculate the sum of the first `k` numbers, then just subtract the number that falls off the left and add the new number on the right.

### Python Solution

```
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) -> float:
        # 1. Get the sum of the very first window of size k
        current_window_sum = sum(nums[:k])
        max_sum = current_window_sum

        # 2. Slide the window from index k to the end of the array
        for i in range(k, len(nums)):
            # Add the new element on the right, subtract the old element on the
            current_window_sum = current_window_sum + nums[i] - nums[i - k]

            # Update the max
            max_sum = max(max_sum, current_window_sum)

        return max_sum / k
```

## Level 2.5: The "Reverse" Sliding Window

**LeetCode 1423: Maximum Points You Can Obtain from Cards**

- **The Goal:** Take exactly $K$ cards from the edges (left or right) to maximize your score.
- **The Trick:** If you take $K$ cards from the outside, you are leaving behind a contiguous subarray of size `N - K` on the inside. To MAXIMIZE your score, you just use a sliding window to MINIMIZE the sum of the `N - K` cards you leave behind!

### Python Solution

```
class Solution:
    def maxScore(self, cardPoints: List[int], k: int) -> int:
        n = len(cardPoints)
        if k == n:
            return sum(cardPoints)

        window_size = n - k

        # Find minimum window of size (n - k)
        current_window_sum = sum(cardPoints[:window_size])
        min_window_sum = current_window_sum

        for i in range(window_size, n):
            current_window_sum = current_window_sum + cardPoints[i] - cardPoint
            min_window_sum = min(min_window_sum, current_window_sum)

        # The max score is the total sum minus the smallest inner window
        return sum(cardPoints) - min_window_sum
```

## Level 3: The Variable Window (Infosys-Level Boss)

**LeetCode 3: Longest Substring Without Repeating Characters**

- **The Goal:** Given a string `s`, find the length of the longest substring without repeating characters.
- **The Logic:** Your window can grow and shrink dynamically.

1. Expand your right pointer to add letters to your window.
2. Use a Set to track what letters are currently in your window.
3. If you see a duplicate letter, you must shrink your window from the left until the duplicate is removed!

## The Blueprint Strategy

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        char_set = set()
        left = 0
        max_length = 0

        # 'right' pointer expands the window
        for right in range(len(s)):

            # If we find a duplicate, shrink the window from the left
            while s[right] in char_set:
                char_set.remove(s[left])
                left += 1

            # Add the current unique character to our set
            char_set.add(s[right])

            # Calculate the size of our current valid window
            current_window_size = right - left + 1
            max_length = max(max_length, current_window_size)

        return max_length
```